

DSAII- HW11

Tomas Salaz

April 22, 2026

Problem 1

To show the amortized cost of the stack and its operations, we can use the potential method.

Let D_i be the stack after operation i , and let Φ be the potential function, where $\Phi = \text{Num Items on Stack } D_i$. $\Phi_0 = 0$ and $\Phi_i \geq 0$ for all i

Since $\Phi_0 = 0$ and $\Phi_n \geq 0$ we have the summations

$\sum_{i=1}^n c_i \leq \sum_{i=1}^n a_i$, which bounds a_i by $O(1)$ and the total real cost by $O(n)$

The Amortized Cost of the i -th operation is given by $a_i = c_i + \Phi_i - \Phi_{i-1}$

With This data struct there are 3 outcomes for any i -th operation: i is a call to `OrderedPush(x)`, i is a call to `Pop()` on a non empty stack, and i is a call to `Pop()` on an empty stack.

Direct Proof

Case 1: Suppose that we `Pop()` $k \geq 0$ items off the stack, then call `OrderedPush(X)`.

The Actual Cost, c_i , is given by the number of calls to each operations, so k pops and 1 push. $c_i = k + 1$.

From here we can calculate the change in potential, $\Phi_i - \Phi_{i-1} = 1 - k$ since it lost k items and gained one. Given all of this we can calculate the amortized cost

$a_i = c_i + \Phi_i - \Phi_{i-1} = (k + 1) + (1 - k) = 2$ Thus, the amortized cost is $O(1)$

Case 2: In any instance, this operations actual cost is $c_i = 1$ since it a simple pop operations. The change in potential is given by $\Phi_i - \Phi_{i-1} = -1$ since we are losing at least one item from the stack. Thus the amortized cost is given by

$a_i = c_i + \Phi_i - \Phi_{i-1} = 1 + (-1) = 0$ Thus the amortized cost is $O(1)$

Case 3: This operation just returns `ERROR` without changing the stack. The actual cost is $c_i = 1$ since we do have to check the stack. The change in potential is given by $\Phi_i - \Phi_{i-1} = 0$ since we neither gain or loose anything from the stack.

Thus the Amortized cost is given by

$a_i = c_i + \Phi_i - \Phi_{i-1} = 1 + 0 = 1$ This the amortized cost is $O(1)$

In any case, the i -th operation is bounded, $a_i \leq 2$, so each operation has amortized cost $O(1)$. Using the summation from the slides $\sum_{i=1}^n c_i = \sum_{i=1}^n a_i + \Phi_i - \Phi_{i-1} \leq 2n = n$, so the total runtime for a sequence of n operations has amortized cost $O(n)$.

Problem 2

Before we start the problems, we need to define Tree T in terms of the colors of its nodes. Let T be a tree rooted at node r , then for each node, v in T , we can define a function to keep track of the Color

$$color(v) = \begin{cases} +1 & \text{if } v \text{ is red} \\ -1 & \text{if } v \text{ is black} \end{cases}$$

For every node $v \neq r$, $p(v)$ = parent node of v . For every path in $v_1, v_2, \dots, v_k$, Let $S_j = \sum_{i=1}^j color(v_i)$. So S_j is the running count along the path to v . Every path is unique. The path is valid IFF $S_j \geq 0$ for every j from 1 to k . Let $f(v) = S_k$ where k is the depth of v Or $-\infty$ Otherwise

Part A: For the recurrence, we define the **Base Case**, $v = r$. The path ending at r is just the root, and thus is valid IFF, $color(r) = 1$, Hence

$$f(r) = \begin{cases} 1 & \text{if } color(r) = 1 \\ -\infty & \text{if } color(r) = -1 \end{cases}$$

For the Recursive case, $v \neq r$, the unique path ending at v is obtained by appending v to the end of the path given by the parent, $p(v)$. 2 things must hold during every iteration:

1. Every Prefix before v must be ≥ 0 , thus $f(p(v)) \neq -\infty$.
 2. The prefix ending at v must hold that $S_k \geq 0$, but $S_k = f(p(v)) + color(v)$ thus, $f(p(v)) + color(v) \geq 0$.
- If both hold, then $f(v) = f(p(v)) + color(v)$ otherwise there is no valid path so $f(v) = -\infty$. So All together the recurrence is

$$f(v) = \begin{cases} 1 & \text{if } v = r, color(r) = 1 \\ -\infty & \text{if } v = r, color(r) = -1 \\ f(p(v)) + color(v) & \text{if } v \neq r, f(p(v)) \neq -\infty \text{ AND } f(p(v)) + color(v) \geq 0 \\ -\infty & \text{otherwise} \end{cases}$$

Part B: The Dynamic Program Algo is described below

FindValidPath(T)

```
//1.Process Tree with Color Value, using BFS
if color(r) = 1    f[r] = 1                                //Root Cases
else              return -∞

For each child, v, from current node u:
    if f[u] = -∞ f[v] = -∞                                //Child Cases
```

```

else:
    next = f[u] + color(v)
    if next >= 0:    f[v]=next
    else            f[v] =  $-\infty$ 

//2.Look for a valid leaf
For each leaf l:
    if f[l]  $\neq -\infty$ :
        //3.Reconstruct the Path from the leaf node
        path[] = empty
        u = l
        while u  $\neq$  NULL:
            append u to path
            u = p(u)
        return (reverse(path))

return  $-\infty$  //No Valid Path exists

```

Correctness and Runtime of Algo

For Step 1, BFS guarantees that we calculate the parent's node color before any node. Thus when we recur, the value $f[v]$ matches the function definition of $f(v)$. Step 2 is correct by the definition of the problem, since a valid path exist from root to leaf IFF $f(l) \neq -\infty$. For Step 3, This is correct since the root to node path is unique for every node in the tree, so by reconstructing the path by following the parent pointers to the root, by reversing this list, we get the correct, unique path from root to leaf.

The Runtime for Step 1 takes $O(1)$ time per node since we are doing a BFS, we visit each nose once, so Runtime is $O(n)$. For Step 2 , we scan the leaf nodes, and keep pointers to the value of the parents, so this also take $O(n)$ time. Step 3 is reconstructing the path so the runtime is bounded by the depth of the Tree, so the runtime is $O(\text{depth})$.

All together the runtime is $O(n)$ when we since in the worst case we go to the last node with all values memeoized.

Problem 3

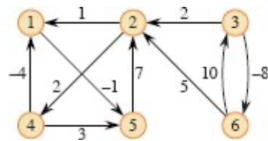


Figure 23.2 A weighted, directed graph for use in Exercises 23.1-1, 23.2-1, and 23.3-1.

The **initial matrix** for the digraph is

$$D^{(0)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & \infty & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & 3 & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{pmatrix}$$

For the **1st iteration**, we use 1 as an intermediate value, which is only useful when going for a path ending at Node 5.

The only update we need to make are to $(2, 5)[2 \rightarrow 1 \rightarrow 5]$ and $(4, 5)[4 \rightarrow 1 \rightarrow 5]$.

$$D^{(1)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & \mathbf{0} & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & \mathbf{-5} & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{pmatrix}$$

For the **2nd iteration**, we allow vertex 2 to be intermediate so we can then update the values for paths that start in the set $[3, 5, 6]$ and end with $[1, 4, 5]$, which is 8 valid pairs, since the diagonals, $(5, 5) = 0$, listed below.

$$\begin{aligned} (3, 1)[3 \rightarrow 2 \rightarrow 1] &= 3 \dots (3, 4)[3 \rightarrow 2 \rightarrow 4] = 4 \dots (3, 5)[3 \rightarrow 2 \rightarrow 1 \rightarrow 5] = 2 \\ (5, 1)[5 \rightarrow 2 \rightarrow 1] &= 8 \dots (5, 4)[5 \rightarrow 2 \rightarrow 4] = 9 \dots (5, 5) = 0 \\ (6, 1)[6 \rightarrow 2 \rightarrow 1] &= 6 \dots (6, 4)[6 \rightarrow 2 \rightarrow 4] = 7 \dots (6, 5)[6 \rightarrow 2 \rightarrow 1 \rightarrow 5] = 5 \end{aligned}$$

$$D^{(2)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ \mathbf{3} & 2 & 0 & \mathbf{4} & \mathbf{2} & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ \mathbf{8} & 7 & \infty & \mathbf{9} & 0 & \infty \\ \mathbf{6} & 5 & 10 & \mathbf{7} & \mathbf{5} & 0 \end{pmatrix}$$

For the **3rd iteration**, we can have 3 as an intermediate value so we can update only the vertex 6 since it has a finite edge to 3. The pairs would start at $[6]$ and end at $[1, 2, 4, 5, 6]$. There is nothing to improve in this step so $D^3 = D^2$.

For the **4th iteration**, we have edges to 1 and to 5, and since the $(4, 1) = -1$ may of the paths improve. The pairs start at $[2, 3, 4, 5]$ and end at $[1, 5]$ they are listed below.

$$\begin{aligned} (2, 1)[3 \rightarrow 2 \rightarrow 4 \rightarrow 1] &= -2 \dots (2, 5)[2 \rightarrow 4 \rightarrow 1 \rightarrow 5] = -3 \\ (3, 1)[3 \rightarrow 2 \rightarrow 4 \rightarrow 1] &= 0 \dots (3, 5)[3 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 5] = -1 \\ (5, 1)[5 \rightarrow 2 \rightarrow 4 \rightarrow 1] &= 5 \dots (5, 5) = 0(\text{Diagnal}) \\ (6, 1)[6 \rightarrow 2 \rightarrow 4 \rightarrow 1] &= 3 \dots (6, 5)[6 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 5] = 2 \end{aligned}$$

$$D^{(4)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ \mathbf{-2} & 0 & \infty & 2 & \mathbf{-3} & \infty \\ \mathbf{0} & 2 & 0 & 4 & \mathbf{-1} & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ \mathbf{5} & 7 & \infty & 9 & 0 & \infty \\ \mathbf{3} & 5 & 10 & 7 & \mathbf{2} & 0 \end{pmatrix}$$

For the **5th iteration**, we allow 5 to be an intermediate value so the start at $[1, 2, 3, 4, 6]$ and end at $[1, 2, 4]$. There isn't much improvement but the pairs that do are listed below.

$$(1, 2)[1 \rightarrow 5 \rightarrow 2] = 6 \dots (1, 4)[1 \rightarrow 5 \rightarrow 2 \rightarrow 4] = 8 \\ (4, 2)[4 \rightarrow 1 \rightarrow 5 \rightarrow 2] = 2$$

$$D^{(5)} = \begin{pmatrix} 0 & \mathbf{6} & \infty & \mathbf{8} & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ 0 & 2 & 0 & 4 & -1 & -8 \\ -4 & \mathbf{2} & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{pmatrix}$$

For the **6th iteration**, we allow 6 to be intermediate which means that we start at $[3]$ and end at $[1, 2, 3, 4, 5]$. The improvements for the pairs are below.

$$(3, 1)[3 \rightarrow 6 \rightarrow 2 \rightarrow 4 \rightarrow 1] = -5 \dots (3, 2)[3 \rightarrow 6 \rightarrow 2] = -3 \\ (3, 4)[3 \rightarrow 6 \rightarrow 2 \rightarrow 4] = -1 \dots (3, 5)[3 \rightarrow 6 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 5] = -6$$

$$D^{(6)} = \begin{pmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ \mathbf{-5} & \mathbf{-3} & 0 & \mathbf{-1} & \mathbf{-6} & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{pmatrix}$$

Problem 4

For each node w , let

$$color(w) = \begin{cases} +1 & \text{if } w \text{ is red} \\ -1 & \text{if } w \text{ is black} \end{cases}$$

For a path $w_1, w_2, \dots, w_k$, let $S_j = \sum_{i=1}^j color(w_i)$.

The black excess of the path is $\max_j (-S_j)$. and the path is still valid IFF its black excess ≤ 0 .

The function $f(u, v, i, b)$ is the maximum of S_k over the path $u \rightarrow v$ whose intermediate nodes have a label $\leq i$ and whose black excess $\leq b$.

Part A: Starting the recurrence with the **Base Case**, $i = 0$, implying that there is no intermediate nodes. This is also the case that $u = v$ and thus, we only need to compute $color(u)$, and check the black excess.

$$f(u, u, 0, b) = \begin{cases} color(u) & \text{if } -color(u) \leq b \\ -\infty & \text{otherwise} \end{cases}$$

If $u \neq v$ and (u, v) is an edge, whose red-black-difference is given by $color(u) + color(v)$. This value is bounded by the black excess. We also need to note that if (u, v) isn't a single edge we return $-\infty$. Thus we get

$$f(u, v, 0, b) = \begin{cases} color(u) + color(v) & \text{if } \text{Max}[-color(u) - color(v), -color(u)] \leq b \\ -\infty & \text{otherwise} \end{cases}$$

For the **Recursive Case**, $i \geq 1$, defines any path from $u \rightarrow v$ with intermediate nodes of $label \leq i$. There are 2 situations, We either use the intermediate node so $u \rightarrow i \rightarrow v$ OR we skip the intermediate node and directly go $u \rightarrow v$. Both cases imply that the intermediate label is bounded, where $label \leq i - 1$.

Now we need to account for the colors. Let $A = f(u, i, i - 1, b)$ and have the best red-black-difference for the 1st sub path, from $u \rightarrow i$. Since i is counted in both sub paths, but only once in the direct path, we have that the combined red-black-difference is $A + \text{Val of 2 sub paths} - color(i)$.

The combined path stays within the black excess bounds IFF the sub path from $i \rightarrow v$ is at most $b + A - color(i)$, thus the best value for the sub path is $f(i, v, i - 1, b + A - color(i))$. SO the entire recurrence is

$$f(u, v, i, b) = \begin{cases} color(u) & \text{if } i = 0, u = v, \text{ and } -color(u) \leq b \\ color(u) + color(v) & \text{if } i = 0, u \neq v, (u, v) \in E \\ -\infty & \text{if } i = 0 \text{ and nothing above holds} \\ \max \begin{cases} f(u, v, i - 1, b), \\ A + f(i, v, i - 1, b + A - color(i)) - color(i) \end{cases} & \text{if } i \geq 1 \end{cases}$$

where $A = f(u, i, i - 1, b)$.

Part B: The Dynamic Program would need to initialize a 4-D Table, $f[u][v][i][b]$, where $u, v \in 1..n$ and $i, b \in 0..n$

We fill in the base case entries, $i = 0$ using the recurrence from above. For $i \geq 1$ and every (u, v, b) , we compute $f[u][v][i][b]$ using the recurrence. In each cell we keep a bit to indicate whether the max was reached routing through i , for path reconstruction. To recover the actual valid path, assuming we never run into a $-\infty$ value, we backtrack through the stored routing bits. We recurse on the subproblems, $f(u, i, i - 1, b)$ and $f(i, v, i - 1, b + A - color(i))$ and splice at i ,

otherwise we recur on $f(u, v, i - 1, b)$.

Correctness: The Program is correct since it exhaustively covers every case where $i = 0$, which are paths with no intermediate nodes. For $i \geq 1$, the DP is correct since the algo covers the 2 mutually exclusive sub problems, Either using the node i to be intermediate, or going directly from $u \rightarrow v$. The reconstruction also get this correct since it has to splice the paths back together at i .

Runtime: The Runtime of the Algo is $O(n^4)$ since the table is size $n * n * n * n$. And since at each entry in the table we are doing $O(1)$ work to complete the entry This the total runtime is $O(n^4 * 1) = O(n^4)$